Neural Networks III

Unsupervised & Reinforcement Learning

Recap

Aspect	Supervised Learning	Unsupervised Learning	Reinforcement Learning
Goal	Learn a mapping from x to y	Discover hidden structure/patterns in x	Learn a policy that maximizes reward
Data	Labeled data	Unlabeled data	State-action pairs, and delayed rewards
Typical output	Classification or regression model	Clusters, low-dimensional embeddings	Policy

Discriminative vs. Generative

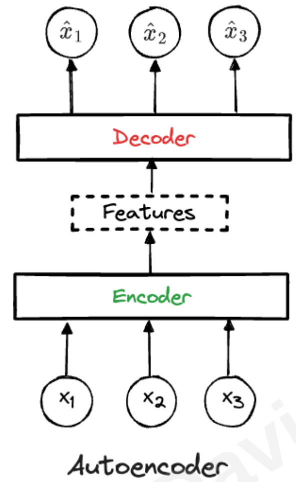
- **Discriminative models**: learn $p(y|x)$, the boundary that best separates classes. **Predict** a label given features.
- **Generative models**: learn $p(x, y)$ or $p(x)$, the full data-generation process. **Generate** new samples and sometimes predict labels by modeling the data distribution.
  Some generative models are trained with unlabeled data (*unsupervised learning*)

Generative Model

- Goal: learn the **data distribution** itself so the model can draw new samples
- Applications
  - Image synthesis, such as DALL·E and Midjourney
  - Music creation, such as Jukebox and Riffusion
  - Super-resolution and photo up-scaling
  - Data augmentation for low-resource tasks.
- Ideas: GANs, diffusion models, deep reinforcement learning

Autoencoder

- Goal: learn a compact representation of inputs and reconstruct them
- Key concepts
  - **Encoder**: compress inputs into hidden features
  - **Decoder**: reconstruct inputs from hidden features
  - **Loss**: make $\hat{x}$ close to x, for example $\|\hat{x} - x\|_2$
- Applications
  - Image compression
  - Anomaly detection (high reconstruction error suggests outlier).
  - ...
- Notable generalization: **variational autoencoder (VAE)**
  Use **variational approximation** to help with sampling
  (More to be discussed in Bayesian analysis)



Link to illustration

Generative Adversarial Network (GAN)

- Goal: generate new samples that look similar to the unlabeled training data
- Starting problem: we only have **real examples**, not labels saying which fake examples are close enough
- Naive idea: generate fake samples directly
- Why that is hard: without feedback, poor generated samples do not tell the generator how to improve
- GAN idea: create the feedback by training another model to judge **real** vs. **fake**

Generative Adversarial Network (GAN)

- A better idea:
  1. Train a model to judge real vs. fake
  2. Train another model to generate samples that fool the judge
- GAN is a duel of two networks:
  - **Discriminator** D: judges real vs. fake
  - **Generator** G: generates fake samples from noise
  - Iterate till convergence
- Training objective

$$\min_G \max_D \{\mathbb{E}_x[\log D(x)] + \mathbb{E}_z[\log\{1 - D[G(z)]\}]\}$$

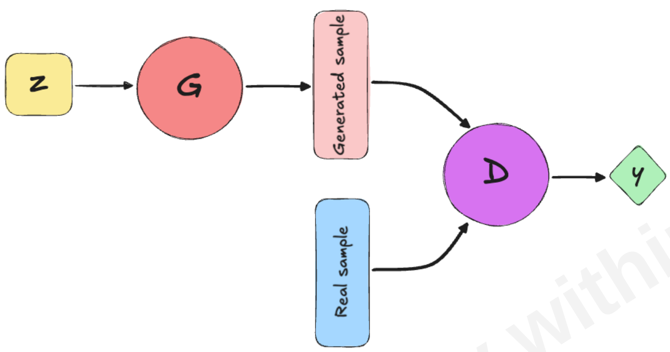
- Original paper: Generative Adversarial Networks

Generative Adversarial Network (GAN)

- Training objective

$$\min_G \max_D \{\mathbb{E}_x[\log D(x)] + \mathbb{E}_z[\log\{1 - D[G(z)]\}]\}$$

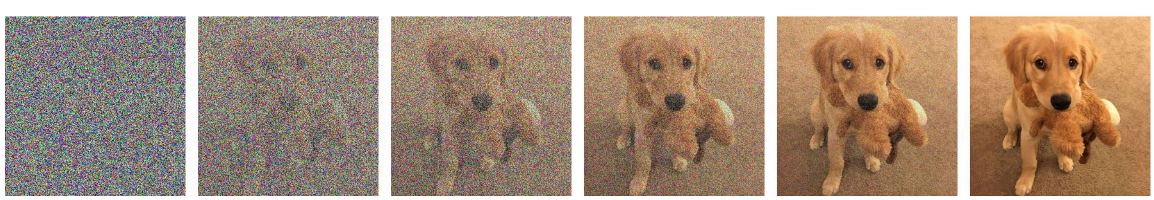
- **Discriminator** wants
  - $D(x) = 1$ for real samples and $D(x) = 0$ for fake samples
- **Generator** wants
  - $D(x) = 1$ for fake samples
- Train D and G using **alternating gradient updates**



Link to illustration

Diffusion Methods

- Goal: generate new samples that are similar to the unlabeled training data
- Modified goal: learn a mapping from **noise** back to the **true sample distribution**
- Idea: train a model to remove a little bit of noise at each step
- Reference: Denoising Diffusion Probabilistic Models and a good visual introduction



Forward noising moves from right to left; generation learns to denoise from left to right.

Reinforcement Learning

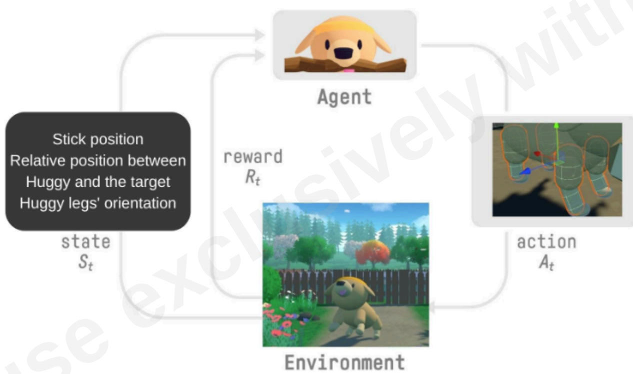
Aspect	Supervised Learning	Unsupervised Learning	Reinforcement Learning
Goal	Learn a mapping from x to y	Discover hidden structure/patterns in x	Learn a policy that maximizes reward
Data	Labeled data	Unlabeled data	State-action pairs, and delayed rewards
Typical output	Classification or regression model	Clusters, low-dimensional embeddings	Policy

Reinforcement Learning

Reinforcement learning (RL) studies how an **agent** should take actions in an **environment** to maximize a **reward signal**. Wiki

- **State** s_t: what the agent observes
- **Action** a_t: what the agent chooses
- **Reward** r_t: feedback from the environment
- Goal: maximize long-term rewards

Training Huggy the dog by HuggingFace:



Two Popular Methods: Intuition

- **Q-Learning**
  - Learn a table of “quality” scores **Q(s, a)** for every *state–action* pair.
  - Keep a cheat-sheet of expected scores for every possible move, then follow the best score.
  - Best for small, discrete tasks
- **Policy-gradient methods**
  - Skip the value table and **tune the policy directly**: a neural network outputs action probabilities.
  - Tune the agent's action probabilities so useful actions become more likely.

- Best for high-dimensional or continuous spaces, or smooth behaviors